
Insulator Defect Identification from Aerial Imagery

Sofia Loya

Department of Mechanical Engineering
Carnegie Mellon University
ssloya@andrew.cmu.edu

Grant Wilkinson

Department of Mechanical Engineering
Carnegie Mellon University
gmwilkin@andrew.cmu.edu

Abstract

This project addresses the detection and classification of insulator defects in aerial power line imagery using the Insulator Defect Identification Dataset (IDID). We compared a baseline model, YOLOv1, to two variants, YOLOv8, across two tasks: detecting full insulator strings and classifying individual shells as good, broken, or flashover damaged. We hypothesized that both YOLOv8 and DINO-DETR would outperform the YOLOv1 baseline, which was partially supported. YOLOv8 outperformed YOLOv1 on both tasks while DINO-DETR outperformed YOLOv1 but did not perform as well compared to YOLOv8's on shell detection.

1 Introduction

Electrical insulators are critical components of power lines, supporting the high voltage wires while preventing current from reaching the support towers Lewis and Kulkarni [2021]. They are constantly exposed to a harsh outdoor environment where damage can easily happen. Due to their location in high places on these towers, damaged insulators can be difficult to identify.

Electric utilities are increasingly relying on high-resolution unmanned aerial system (UAS) imagery and video to inspect transmission and distribution infrastructure. This data is extremely useful for identifying infrastructure defects and prioritizing maintenance decisions. However, while the ability to collect this data has gotten easier, the ability to interpret the quantity of this imagery has fallen behind. Many modern data interpretation solutions still rely heavily on human-in-the-loop review processes that manually review captured imagery to identify defective insulators.

The motivation for this project is to address this bottleneck by leveraging machine learning to automate inspection tasks. Using the Insulator Defect Image Dataset (IDID) dataset from Lewis and Kulkarni [2021], which contains high-quality labeled images of transmission line insulators, our objective is to develop a model capable of detecting defective insulators. The task involves identifying insulator strings and classifying specific defect states of the shells that make up these strings across three classes: *broken shells*, resulting from mechanical stress; *flashover damage*, caused by electrical discharge; and *good shells*, representing healthy infrastructure. Figure 1 shows an example of this labeled dataset.

The count of labeled assets within the IDID dataset are shown in Figure 4 (see Appendix A). For our purposes, we split this labeled dataset into a training, validation, and testing split at a ratio of 70:15:15 respectively.

A successful implementation of this deep learning model would enable higher throughput and automated evaluation of large insulator imagery datasets. It would also ensure consistent, repeatable grading of insulator quality. In the long-term automated identification of defective insulators could enable fully automated inspection workflows where autonomous drones fly along power infrastructure, perform inspections, identify defects, and create maintenance work orders without ever needing a human in the field.



Figure 1: Example labelled image from the IDID dataset. The labelled assets are the insulator string (brown), shells with flashover (red), and good shells (green). Broken shells are not pictured in this image.

2 Methods

2.1 Baseline Model: YOLOv1

For our baseline, we selected the YOLOv1 (You Only Look Once) architecture Redmon et al. [2016]. YOLOv1 is a foundational computer vision model designed for object detection. It is a highly appropriate baseline for the IDID dataset because it treats object detection as a single regression problem, mapping image pixels directly to bounding box coordinates and class probabilities.

2.1.1 How the Baseline Model Works

YOLOv1 divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that cell is responsible for detecting that object. Each grid cell predicts B bounding boxes and confidence scores for those boxes. These confidence scores reflect how confident the model is that the box contains an object and how accurate it thinks the box is. Simultaneously, each cell predicts C conditional class probabilities. The final output is a tensor of shape $S \times S \times (B \times 5 + C)$.

For the IDID task, we developed two different YOLOv1 models. One model detects the overall insulator string. The second model detects the three different classes of individual insulator shells that make up the insulator strings: *broken*, *flashed*, and *good*.

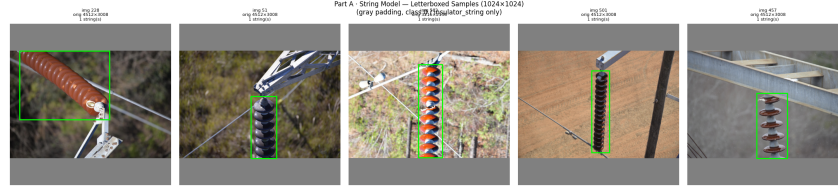
The model is optimized using a four-part loss function. This includes a localization loss for bounding box coordinates, an object confidence loss for cells containing objects, a no-object confidence loss to penalize false positives in the background, and a classification loss for the specific defect types. To balance these terms, the localization loss was weighted more heavily ($\lambda_{coord} = 5$), while the no-object confidence loss was down-weighted ($\lambda_{noobj} = 0.5$).

2.1.2 Assumptions and Suitability

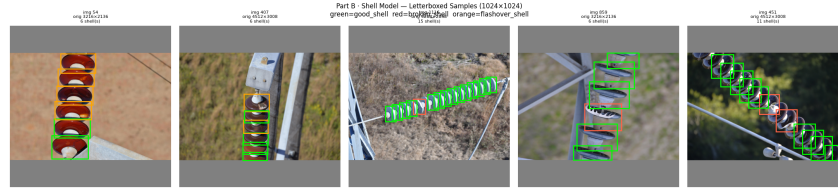
YOLOv1 makes several key assumptions about the data:

- **Spatial Constraints:** It assumes that each grid cell can only contain one object. This can be a limitation for overlapping objects, which is why we trained two different models - one for the insulator string and one for the shells.
- **Global Context:** YOLOv1 sees the entire image during training and testing, allowing it to encode contextual information about the transmission lines and towers. This reduces background false positives like mistaking a distant tree branch for a broken shell.
- **Input Size:** YOLOv1 requires the input image to be a square image of 448 x 448 pixels. This requires the high-resolution input image to be resized to fit this requirement.

We attempted two methods for this: slicing and letter-box down-scaling. Due to training requirements, letter-box down-scaling was implemented. This means the full image was resized to 448 pixels wide and neutral padding was added to the top and bottom of the image to achieve the square resolution.



(a) Example letter-boxed training input for the insulator strings.



(b) Example letter-boxed training input for the insulator shells.

Figure 2: Example resized and padded inputs for the training loop using a letter-box approach to avoid changing aspect ratios.

YOLOv1 is a logical choice for the IDID dataset because utility inspections require a balance between accuracy and efficiency. Its unified architecture provides a baseline that is significantly faster than R-CNN variants, establishing a performance standard for both the detection of the parent insulators and their defects. Additionally, there is not a need for masks to be created for the insulators. The ultimate goal is to do object detection and classification, for which a classified bounding box is sufficient.

2.2 First Variant Model: YOLOv8

For our first variant model, we selected the YOLOv8 architecture, introduced by Jocher et al. [2023]

2.2.1 How the Variant Model Differs from Baseline

- **Backbone:** YOLOv1 convolution stack is replaced with CSPDarknet, which divides the feature maps into two branches at each stage. One branch passes through a convolution block while the other goes around Wang et al. [2020]. This design preserves feature representation while reducing redundant gradients.
- **Neck:** A PANet feature pyramid aggregates features through both top-down and bottom-up pathways, which enables both high-resolution spatial features from early layers and high-level semantic features from deep layers, something absent in YOLOv1.
- **Detection Head:** A decoupled head separates classification and regression branches into two parallel streams, replacing YOLOv1's single shared feature vector. Box coordinates are predicted using task aligned assignment that matches predictions to ground truth.
- **Loss Function:** MSE is replaced by three components: CIoU for box regression, Distribution Focal Loss (DFL) for modeling box edge distributions Li et al. [2020], and binary cross-entropy for classification.

2.2.2 Choice of Variant Model

YOLOv8 was chosen as it is a more recent generation of the YOLO object detection architecture. YOLOv8 has demonstrated strong performance across a variety of detection tasks, particularly in scenarios involving multi-scale objects, class imbalance, and high-resolution imagery Jocher et al. [2023], which are all present in the IDID dataset.

2.3 Second Variant Model: DINO-DETR

For our second model variant, we selected the DINO-DETR (DEtection Transformer) architecture, introduced by Zhang et al. [2023]

2.3.1 How the Variant Model Differs from Baseline

- **Use of Transformers:** Unlike YOLO which is a CNN-based model, DINO utilizes a transformer based approach. This enables it to have global attention during inference.
- **Larger Input Size:** DINO is multi-scale and accepts inputs with varying resolutions. This enables us to increase the size of the inputs to 1024 x 1024. There is not actually a need to make the images square, and a different aspect ratio could be used. However, for consistency with the YOLO approaches, a letter-boxed approach with padding was used for DINO as well.
- **DINO Backbone:** This model makes use of Meta’s DINOv3 backbone to extract the features from the input images, introduced by Siméoni et al. [2025]. This is a self-supervised imagery foundational model meant to be applicable to many different image analysis tasks. This is a variation in architecture of the model presented in the DINO-DETR paper.

2.3.2 Choice of Variant Model

The DINO-DETR model was chosen as it is also an object detection model that is built from a newer generation of transformer-based models. In other studies cited in their paper, the DINO backbone has proven capable in a variety of applications, from image segmentation, classification, and, of course, object detection, which is required to analyze the IDID dataset.

2.4 Image Processing Approach

We attempted two different image processing approaches. Initially, we planned to have the insulator string be detected from the down-scaled full image while the shell detection would utilize SAHI tiles to have a higher resolution, as demonstrated by Akyon et al. [2022]. We hypothesized that the entire image would be necessary for the string class, as it was a global feature of the image. At the same time, we thought that SAHI tiles would lead to better performance on the smaller shell classes as it would give the detector a lot more detail about each individual shell within the tile image as opposed to the global image Figure 7 (see Appendix B).

Unfortunately, the large increase in training requirements associated with training on the order of magnitude more images created by SAHI was not feasible. During our tests, we were not getting the models to converge within a reasonable amount of time. The results we did get, while directionally correct and promising with more training time, were not satisfactory. Figure 7 shows the large number of extraneous detections from the tiling output. Given more time, this model could likely be significantly improved and the cluttered inference could be fixed through a combination of improved model training and better merging of bounding boxes.

Ultimately, given that we were seeing good quality results with the alternative down-scaling approach, we decided to proceed with that for both the string and the shell detections. This cut the required training time for the models by nearly an order of magnitude.

3 Results and Discussion

3.1 Model Training Performance

We trained all both models from both model variants using Google Colab. The YOLOv1 model was trained from scratch, following the paper’s architecture. The YOLOv8 models utilized Ultralytics for model training. And the DINO-DETR models fine-tuned the DINO model and trained the DETR head from scratch. We utilized mean Average Precision at an IoU threshold of 0.50 (mAP@50) as our primary evaluation metric. mAP@50 was selected because it evaluates both localization accuracy and classification accuracy. The results in terms of mAP@50 are shown in Table 1.

Table 1: Performance Comparison of Object Detection Models on IDID Dataset

Model	String Detection (mAP@50)	Shell Detection (mAP@50)
YOLOv1	0.5869	0.0101
YOLOv8	0.9950	0.9276
DINO-DETR	0.8182	0.3063

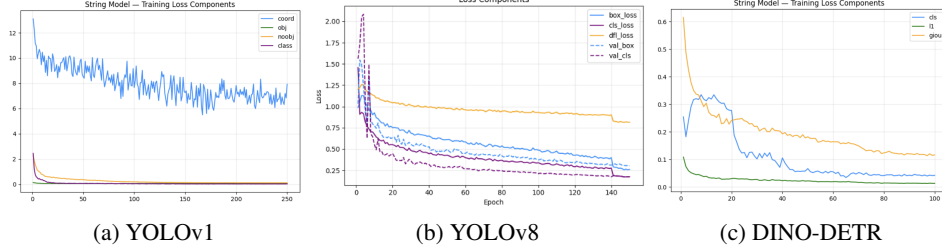


Figure 3: Training loss on the overall string model shown for the three model variants.

Examples of the training loss for the three model variants is shown in Figure 3 for further detail refer to Figure 5, 6, 8 (see Appendix B-C). The pictured loss plots are for the insulator strings. As can be inferred from the results in Table 1, the YOLOv8 variant was the best converging model. For string detection, YOLOv8 achieved mAP@50 of 0.9950 compared to 0.5869 for YOLOv1. For shell detection, where YOLOv8 achieved 0.9276 compared to 0.0101 for YOLOv1.

DINO-DETR also converged relatively well compared to the baseline YOLOv1. DINO-DETR achieved 0.8182 on string detection and 0.3063 on shell detection. While string detection performance is good, the shell detection result of 0.3063 shows that the transformer-based approach benefits less.

We believe that one of the things that was most difficult for the YOLOv1 approach to handle was the long slender nature of the insulator strings. YOLOv1 tends to perform best on relatively square objects where the object mostly fills up the bounding box. However, the strings often result in a large portion of the in-box area being background that seemed to confuse especially the older model.

Our hypothesis that both YOLOv8 and DINO-DETR would outperform the YOLOv1 baseline was partially supported by the results. Both variants outperformed YOLOv1 in both detection tasks, confirming the general hypothesis. However, DINO-DETR’s performance was weaker than expected, particularly on shell detection, where it achieved only 0.3063 mAP@50 compared to YOLOv8’s 0.9276. DINO-DETR generally requires more training data than what was available for the IDID dataset. The self-attention mechanism would have specifically benefited from a larger data size because it would have been exposed to a larger variety of examples.

4 Conclusion

This project compared YOLOv1, YOLOv8, and DINO-DETR for insulator defect detection across two tasks: string detection and shell classification. Our hypothesis that both YOLOv8 and DINO-DETR would outperform the YOLOv1 baseline was partially supported. Both variants did outperform YOLOv1, however DINO-DETR performed poorly compared to YOLOv8 in shell detection.

The most surprising finding was YOLOv1 poor performance on shell detection. This was likely due to the combination of small object size, class imbalance, and YOLOv1’s single-scale grid architecture. Additionally, we expected DINO-DETR to perform better on shell detection given its more modern transformer-based architecture. With only 1,120 training images, DINO-DETR may not have had sufficient data to fully leverage its transformer backbone, while YOLOv8’s convolutional inductive biases allowed it to generalize more effectively from the smaller dataset.

Given more time and compute, we would investigate SAHI-based tiling for shell detection. Initially we planned to use this approach but stopped due to training time constraints and limited compute. With more compute this approach could significantly improve small object detection across all three models.

Collaboration Statement

Sofia and Grant collaborated on the YOLOV1 model, while Sofia completed the YOLOv8 model and Grant completed the DINO-DETR model. AI was used to troubleshoot errors in the code base, for assistance in the implementation of some training architecture, and for editing in the report.

References

- Dexter Lewis and Pratik Kulkarni. Insulator defect detection, 2021. URL <https://dx.doi.org/10.21227/vkdw-x769>.
- Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- Glenn Jocher, Ayush Chaurasia, and Jing Qiu. YOLO by Ultralytics, 2023. URL <https://github.com/ultralytics/ultralytics>. Available: <https://github.com/ultralytics/ultralytics>.
- Chien-Yao Wang, Hong-Yuan Mark Liao, Yueh-Hua Wu, Ping-Yang Chen, Jun-Wei Hsieh, and I-Hau Yeh. CSPNet: A new backbone that can enhance learning capability of CNN. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 390–391, 2020.
- Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems*, volume 33, pages 21002–21012, 2020.
- Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel M. Ni, and Heung-Yeung Shum. DINO: Detr with improved denoising anchor boxes for end-to-end object detection. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arxiv.2203.03605. URL <https://openreview.net/forum?id=3mRwyG5one>.
- Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. Dinov3. *arXiv*, abs/2508.10104, 2025. doi: 10.48550/arxiv.2508.10104.
- Fatih Cagatay Akyon, Onur Can Altun, and Alparslan Aktas. Sahi: A generic framework for zero-shot object detection and instance segmentation. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 2631–2635. IEEE, 2022. doi: 10.1109/ICIP46576.2022.9897990.

Appendix

A. IDID dataset

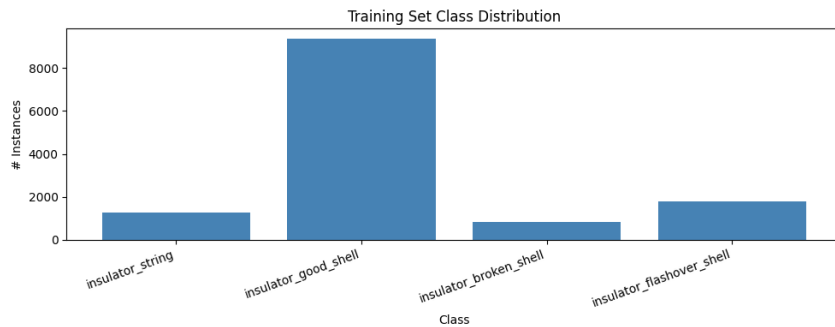


Figure 4: Distribution of classes within the overall labeled IDID dataset.

B. YOLOv1

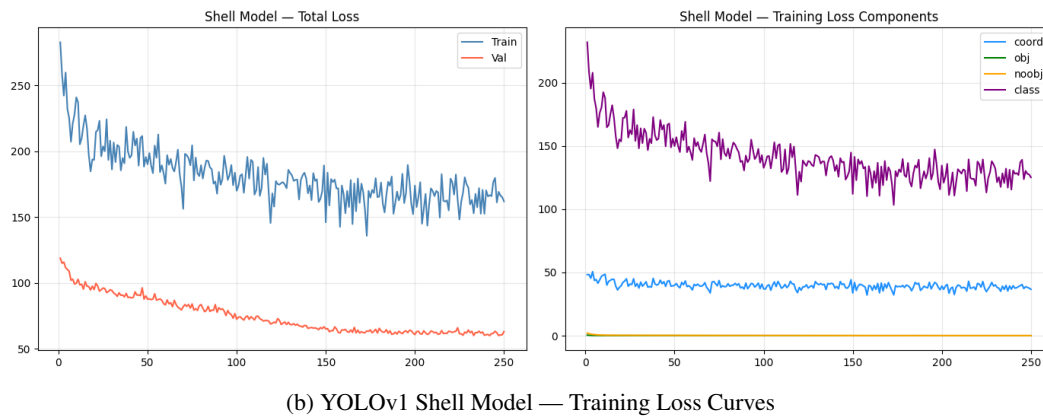
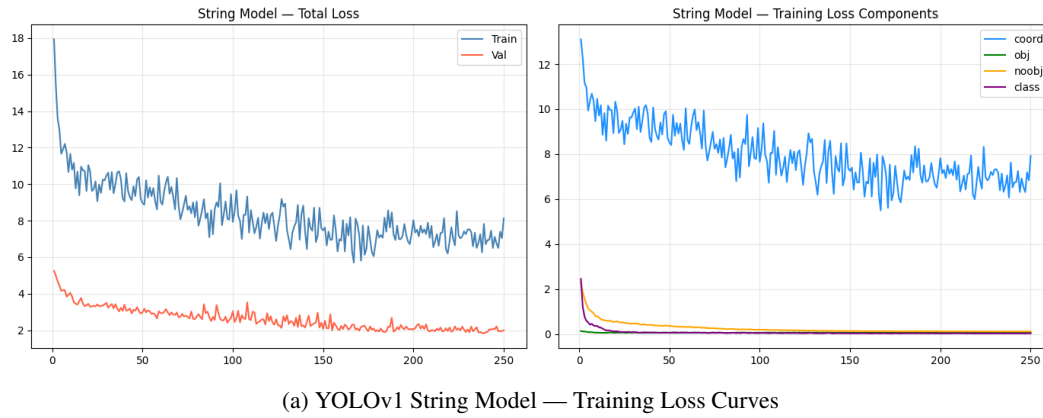
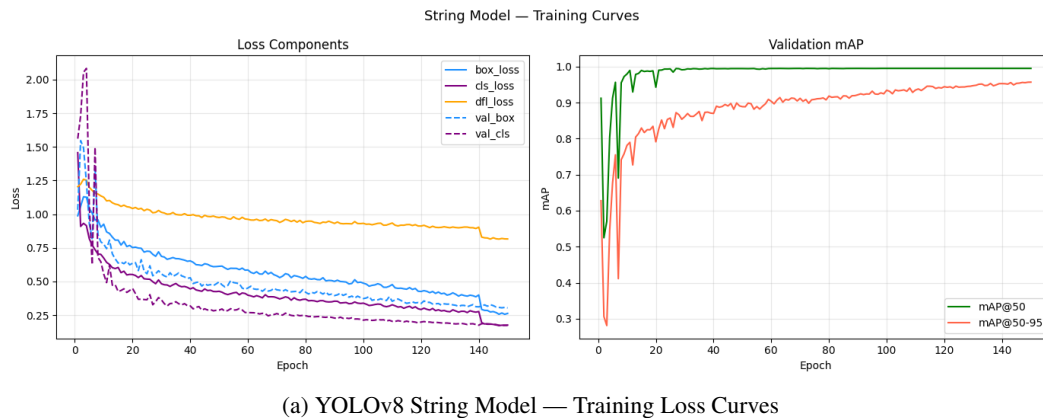
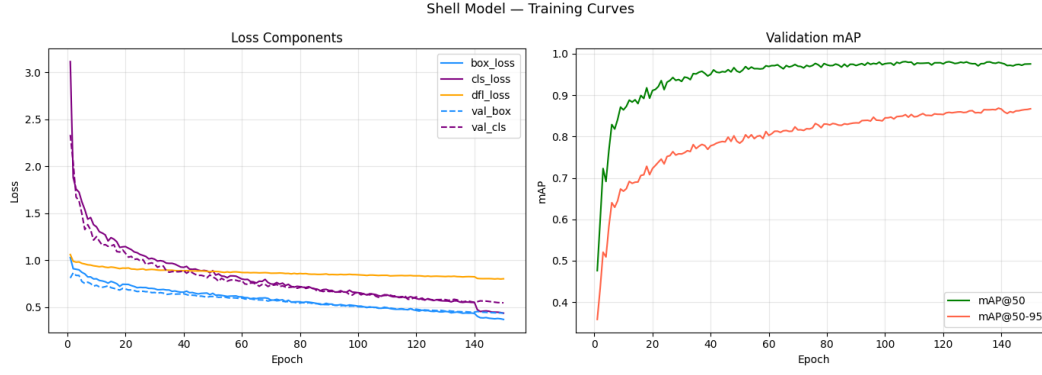


Figure 5: YOLOv1 training loss curves for both the string detection model (top) and shell detection model (bottom). Both models struggled to converge using the YOLO architecture.

C. YOLOv8





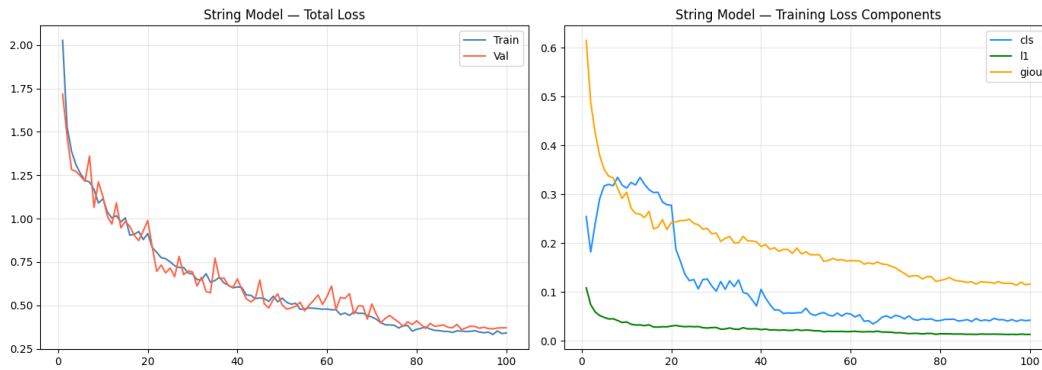
(b) YOLOv8 Shell Model — Training Loss Curves

Figure 6: YOLOv8 training loss curves for both the string detection model (top) and shell detection model (bottom). Both models show consistent convergence over 150 epochs.

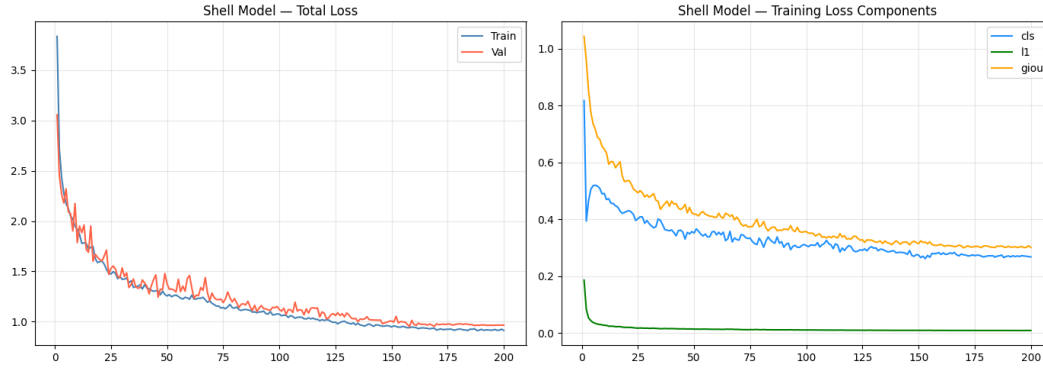
D. DINO-DETR



Figure 7: Example inference output of tiling-based detection of insulator shells. The correctly labeled image is on the left, the predicted labels are on the right.



(a) DINO-DETR String Model — Training Loss Curves



(b) DINO-DETR Shell Model — Training Loss Curves

Figure 8: DINO-DETR training loss curves for both the string detection model (top) and shell detection model (bottom).

E. Code

Code is available on GitHub: <https://github.com/gmwilkin/deep-learning-project.git>